

MSDM 5001

An Introduction to Computational and Modelling Tools

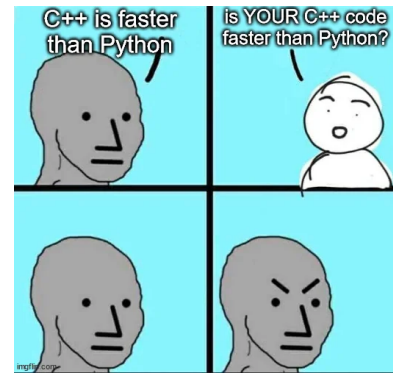
Lecture 9: Introduction to numpy, numba and Cython, Part II

Reminder: Installation

- Install relevant packages:
 - `uv pip install numpy numba cython setuptools jupyter ipykernel`
 - To use Cython, you need to make sure you have a C/C++ compiler properly installed on your system

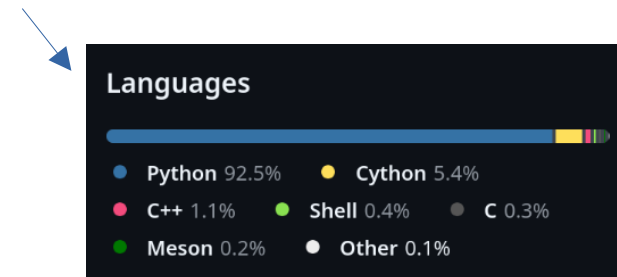
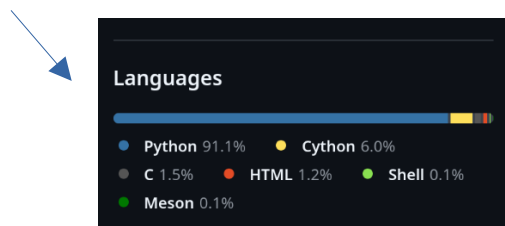
The two-language problem

- “Simple” languages (e.g. Python)
 - Easy to code in, less development time
 - Slow and inefficient at runtime
- “Difficult” languages (e.g. C/C++)
 - Difficult to code in, more error-prone (?), more development time
 - Fast and efficient at runtime (if coded well...)
- One solution: JIT compilation
 - Julia
 - numba



ython: A super set of Python

- Compiled into extension modules, which can be called in Python
- Can also be used to wrap existing C/C++ libraries
- Static typing
- A mix of C-like and Python-like syntax (some hate it)
- Great for numerical and scientific computing
- Great integration with numpy workflow
- Used in a lot of scientific computing libraries like scikit-learn, scipy, pandas etc.



Aside: what are extension modules?

- Recall:
Python code
 - Bytecodes
 - Python VM reads and translate on the fly
- Extension modules
 - C code pre-compiled into machine code callable by Python VM
 - Python VM no longer interprets bytecodes
 - Fast!

A taste of Cython's performance



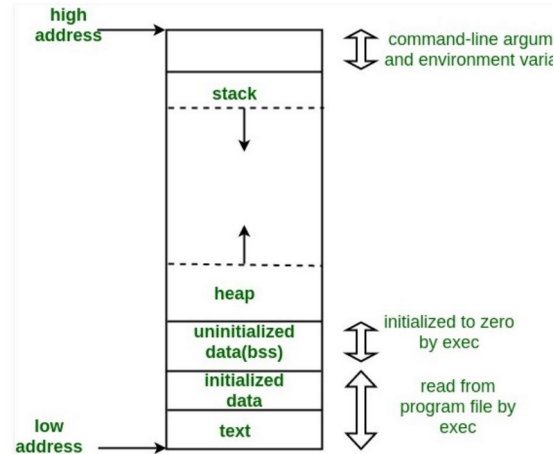
		n = 100	Speed-up
Python	<pre>def func(n): a, b = 0.0, 1.0 for i in range(n): a, b = a + b, a return a</pre>	~ 8 μ s	1
C++	<pre>double func(int n) { int i; double a=0.0, b=1.0, tmp; for (i=0; i<n; ++i) { tmp = a; a = a + b; b = tmp; } return a; }</pre>	~ 145 ns	~ 55
Cython	<pre>def func(int n): cdef int i cdef double a=0.0, b=1.0 for i in range(n): a, b = a + b, a return a</pre>	~ 155 ns	~ 50

What makes it so fast?

- Interpreted vs compiled
 - Interpreter adds overhead
- Static vs dynamic typing
 - Runtime type checking → overhead
- Stack vs heap
 - Almost all Python objects (e.g. Dictionary, List, floats) are heap allocated
 - Heaps are much slower than stack

More on stack vs heap

- Typically a program is allocated two types of memory: stack and heap (don't confuse them with data structures of the same name)
- Stack
 - For primitive variables (e.g. ints, floats) and function call info
 - Allocation and deallocation are automatic and fast
 - LIFO
- Heap
 - For objects and persistent data
 - Requires manual memory management (in Python, this is done by reference counting)
 - Slow access



Not all sunshine and rainbows...

- You are not always guaranteed performance boost by rewriting your code into Cython, which sometimes can be quite an effort!
- When does Cythonizing not work?

Compiling Cython codes

- In Jupyter, use magic commands `%load_ext Cython` and `%%cython`
 - Fast for prototyping
 - What I will doing for most of the lecture
- Recommended: `setup.py`

setup.py

- A file defining how compilation is done

```
from setuptools import setup, Extension
from Cython.Build import cythonize
import numpy as np

extensions = [
    Extension(
        "cool_extension",
        ["cool_extension.pyx"],
        include_dirs=[np.get_include()],
        extra_compile_args=["-O3"],
        extra_link_args=[],
        language="c++"
    )
]

setup(
    name="cool_extension",
    ext_modules=cythonize(
        extensions,
        compiler_directives={
            'language_level': "3",
            'boundscheck': False,
            'wraparound': False,
        }
    ),
)
```

name used when importing
source files
header files directories
compiler flags
linker flags
use c++ compiler

use Python 3 syntax
disable bounds checking
disable negative indexing

Compiling Cython files with setup.py

```
# Build extension modules in-place (development)  
python setup.py build_ext --inplace  
  
# Install package into environment  
python setup.py install  
  
# Build distribution  
python setup.py sdist  
  
# Clean build artifacts  
python setup.py clean --all
```

- See demo

Declaring types in Cython - `cdef`

- To statically type variables, use `cdef <type>`
- Supported types

- C types {
- Numerics: `char`, `int`, `long`, `float`, `double` (and unsigned versions), ...
 - Boolean: `bint`
 - Pointers: `int*`, `double*`, etc.
 - Stack-allocated C-arrays: `int[10]`, `double[10][20]`
 - Function pointers: `void (*f)(int, double)`
 - Built-in Python types: `list`, `dict`, `str`, `set`, ... ← Still Python objects after `cdef`!!!

Note: Cython does not support using `*` for dereferencing pointers, use index-at-zero instead

Declaring types in Cython - cdef

- You can mix statically and dynamically typed variables!

```
cdef double x, y, z  
# some calculations  
coordinates = (x, y, z)
```

- Rule of thumb:
“More static typing → better chance of optimization”

cimport

- Imports C-level info at compile time
 - C-types, C functions defined in C libraries

```
from libc.math cimport sin, cos
from libc.stdlib cimport malloc, free
```

- Cython-specific declarations

```
from cython cimport boundscheck, wraparound
cimport cython
```

- If you know C/C++, it's like #include
 - How is it different from regular import?

Python functions vs C functions

- Python functions
 - First class objects: can be created and returned inside other functions, and can be passed as arguments directly
 - Slow due to Python interpreter overhead
- C functions
 - Can be passed as argument (function pointers), but much more cumbersome
 - Takes arguments only by positions
 - Fast!

Three kinds of functions in Cython

- `def`
 - Normal Python functions, can optionally type arguments
 - Slow
- `cdef`
 - Preferably only deal with C types
 - Fast (if only C types are involved)
 - Cannot be called by external Python codes
- `cpdef`
 - Can only accept and return variables compatible with both Python and C
 - Can be called from external Python codes
- See demo

Looping in Cython

- For efficient looping, cdef the loop variables

```
cdef i, n
for i in range(n):
    # loop
```

- Warnings
 - When looping over containers (e.g. list, dict, tuple), typing the index variables may lead to even more overhead
 - Consider (i) converting the container to equivalent C++ containers (won't be covered), or (ii) using typed memoryviews (see later)
 - If possible, use range

Exercise 1: Trapezoidal Rule

- Integrals can be numerically estimated by the trapezoidal rule:

$$\int_a^b f(x) \, dx \approx \frac{\delta}{2} \sum_{i=0}^{n-1} [f(x_i) + f(x_{i+1})] \quad \delta = \frac{b-a}{n}, \quad x_i = a + i\delta$$

- Implement this as a function `trapezoidal` in Cython
 - It should accept 4 arguments: a function pointer, two doubles, and an int
 - It should return a double
- Implement the same function using numpy
- Test out your implementations with some arbitrary functions. Which is faster?

Python's buffer protocol

- An interface that exposes an object's internal memory layout *directly* to other objects *without copying the data* (c.f. accessing an element in a Python list)
- Allow objects to share memory → efficiency!
- Types that support buffer protocol
 - numpy array
 - memoryview ← `memoryview(obj)` creates a view into the memory of an object that supports the buffer protocol, see demo
 - array.array
 - arrays in ctypes

Cython's typed memoryviews

- Similar interface to Python's memoryview
- C-level type (compared to Python's native memoryview)
 - Again, bypasses Python overhead
- Declaration:

```
cdef double[:]  
cdef double[:, :]  
cdef double[:, :, ::1]  
cdef double[:, ::1, :]
```

1D memoryview of doubles
2D memoryview of doubles
3D, C-contiguous memoryview of doubles
3D, F-contiguous memoryview of doubles

Two examples

- Which is faster?

```
def func1(double[:] mv):  
    cdef double x, sum = 0.0  
    for x in mv:  
        sum += x  
    return sum
```

```
def func2(double[:] mv):  
    cdef:  
        double sum = 0.0  
        int i, N  
  
    N = mv.shape[0]  
    for i in range(N):  
        sum += mv[i]  
    return sum
```

- What will happen?

```
arr = np.arange(100).reshape(10, 10)  
cdef int[:, ::1] mv = arr[:, ::2]
```

Turning off bounds checking & wraparound

- By default, every time we index into a memoryview, Cython checks whether the index is within bounds.
- It also allows negative indexing (“wraparound”)
- But these safety measures come at performance cost
- You can turn them off

```
# cython: boundscheck=False  
# cython: wraparound=False
```

← At the top of file

or

```
from cython cimport boundscheck, wraparound
```

```
def func(double[:] mv):  
    with boundscheck(False), wraparound(False):  
        # something
```

```
@boundscheck(False)  
@wraparound(False)  
def func(double[:] mv):  
    # something
```

Exercise 2: Game of life (Cython)

- Translate the numba version of Game of Life from Lecture 8 into Cython.
- How does the performance compare against your numpy version?

Parallel programming in Cython

- Reminder: (C)Python has GIL before 3.14
- Cython has the ability to turn off the GIL (under some conditions)

```
cdef func() nogil:  
    # ...
```

← cannot have any Python objects

“this function can be called without the GIL”

- However, we still have to write parallel codes ourselves.
- We cannot just use the threading module before 3.14 even with nogil functions (why?)

Parallel programming in Cython

- OpenMP
 - API to do shared-memory parallel programming in C/C++/Fortran
- Cython exposes some of the constructs in OpenMP
 - Most useful to us: prange
 - For parallelizing loops:
YOU are declaring the loops can be run in parallel



prange in action

```
import numpy as np
from cython.parallel cimport prange
from libc.math cimport sin

cpdef double[:] vec_sine_add(double[:] arr1, double[:] arr2):
    cdef int i, n = arr1.shape[0]

    cdef double[:] arr3 = np.empty(n)

    with nogil:
        for i in prange(n):
            arr3[i] = sin(arr1[i]) + sin(arr2[i])

    return arr3
```

Using prange

- In setup.py:

```
extensions = [  
    Extension(  
        "cool_extension",  
        ["cool_extension.pyx"],  
        extra_compile_args=['-fopenmp'],    # Enable OpenMP  
        extra_link_args=['-fopenmp'],  
        include_dirs=[np.get_include()]  
    )  
]
```

- In Jupyter:

```
%%cython --compile-args=-fopenmp --link-args=-fopenmp
```

- See demo

Exercise 3: Mandelbrot Set

- Recall the Mandelbrot set:

$$\{c \in \mathbb{C} : f(z) = z^2 + c \text{ does not diverge upon iterations starting with } z = 0\}$$

- Generate it using Cython
- Idea: For each pixel on the screen, determine if it is in the Mandelbrot set, if yes, set the output value to 1, else 0.
- Note: This is embarrassingly parallel across pixels! Use prange!
- You likely won't get perfect scaling with prange, why?

References

- Cython's official webpage:
<https://cython.org/>
- Cython - a guide for Python programmers, by Kurt W. Smith